



MATES Computer Science

Senior Capstone Project Bi-Weekly Progress Report

Project Title	Olive the World
Team Members	Zachary Ciappa , Ella Jacinto , and Sabrina Shi
Dates Covered by Report	February 2, 2026 - March 2, 2026
Link to Github	https://github.com/jackandtoes/Olive-the-World

1. Summary of Project

Olive the World! is a web game consisting of a variety of minigames. Players will travel around the world as Olive the olive with the goal of becoming the best chef on the planet. Each minigame is modeled after an existing popular game; the Philippines game resembles Crossy Road, Italy is inspired by Pipe Mania, and Mexico is modeled after Fruit Ninja. Olive the World! will be free for public use, opening minds to diverse cuisines from around the world.

2. Summary of Progress this Period

During this progress period, a playable and interactive framework for each game was created. Additionally, all of the games were connected to each other, each being contained within its own scene. The use of GitHub has been extremely useful for working independently but still together. For the Philippines game, the map was created and the collision function was added to the game. This also resulted in a win/lose function being implemented in the game. For the Italy game, the grid background, start, and end of the pipes was created. The pipes were also added manually with the addPipes function. This game is easily customizable in VS code, and it is interactive. The Mexico game also has an established framework, consisting of rectangles flying in the air and counting how many are collected. It also has a built-in countdown for the time limit. All team members are still working on their codecademy courses. This course is more useful for some members compared to others, so online tutorials and Reddit threads are very useful. We continue to work as a team to look at each others' games, giving feedback and proposing improvements.

3. Detailed Progress this Period, separated by Team Member Zac's Accomplishments

I am 25% done with the "Learn Game Development with Phaser.js" CodeCademy course, which is a 7 hour course. The first part of this course is learning Phaser basics, and covers most of the logic and code I need to use for my game.

I have also found code that is useful for my minigame ([source](#)). I used AI coding assistance to adapt this to the specific format we needed to use for our game. I was able to code an "addPipes" function that added in pipes at specific locations for you to connect.

```

addPipes() {
  let bentX = [2,2,3,6,2,1,4,3,5,3];
  let bentY = [4,5,3,4,0,6,4,5,5,7];

  for (let i = 0; i < bentX.length; i++) {
    this.createPipe(bentX[i], bentY[i], 'bent');
  }

  let straightX = [1,2,2,2,1,3,4,6,3,5];
  let straightY = [4,1,3,2,0,4,6,7,6,6];

  for (let i = 0; i < straightX.length; i++) {
    this.createPipe(straightX[i], straightY[i], 'straight');
  }

  let crossedX = [2, 4];
  let crossedY = [7, 5];

  for (let i=0; i < crossedX.length; i++) {
    this.createPipe(crossedX[i], crossedY[i], 'crossed')
  }

  let t_shapeX = [4,4,7,5];
  let t_shapeY = [2,7,1,7];

```

I also created little pasta noodles in Canva that you can spin. I have placed different noodle types in different areas on the grid, creating a playable game. AI coding assistance also helped me add animation to these noodles, which looks very aesthetically pleasing.

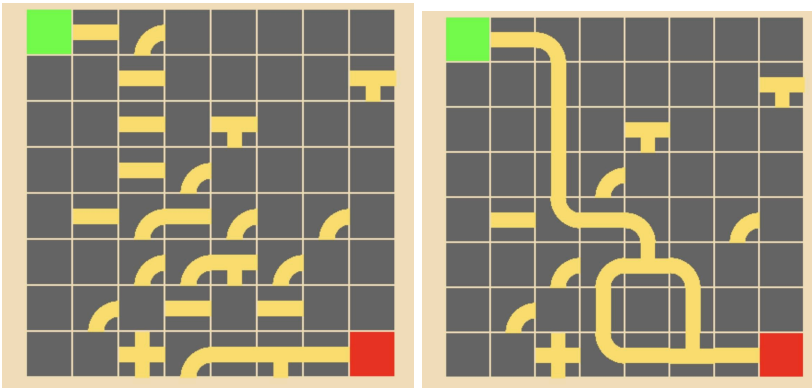
```

const pipe = this.add.image(
  gridX * this.GRID_SIZE + this.GRID_SIZE/2,
  gridY * this.GRID_SIZE + this.GRID_SIZE/2,
  textureKey
);

pipe.setDisplaySize(
  this.GRID_SIZE,
  this.GRID_SIZE);
pipe.setInteractive();

pipe.rotation = Phaser.Math.DegToRad(tile.rotationIndex * 90);
let pointer_count = 0;
pipe.on("pointerdown", () => {
  tile.rotationIndex = (tile.rotationIndex + 1) % 4;
  pointer_count += 1;
  this.tweens.add({
    targets: pipe,
    rotation: pipe.rotation + Phaser.Math.DegToRad(90),
    duration: 200
  });
});

```



We have linked this set of code together with the rest of our minigames, which is the start of a completely playable game. Another thing that I have done during this period is get used to the GitHub workflow. We were able to link our code in VS code to our Github repository, and I have found it very useful. I have created my own branch that I will work on (italy-minigame), and I have done pull requests and merged with the main branch. This was definitely a learning experience, but it is extremely useful now that I know how to do it.

Ella's Accomplishments

I am continuing the 7 hour CodeCademy course on Game Development using Phaser and I am currently 56% done with it. The first module about using Phaser basics was extremely helpful as it showed me how to implement basic keyboard controls to control a character on the screen. I was able to use that knowledge to start the framework for the level selection code and my own minigame code. The third module about learning Phaser animations was also very useful because it taught me how to implement multi-scened games which helped me understand how to link all of our minigames together as well as the level selection code using the constructor method to initialize the Scene objects.

Additionally, I found a remake of a 2D version of Crossy Road (without using Phaser) and I have taken inspiration from the GitHub Repository: [2D-Crossy-Road](#)

The repository code (left) helped me implement grid-based player movement by showing how to snap the player position to tiles. I adapted this logic into my own code (right), which made player movement more consistent and easier to manage.

```
function moveOlive() {
  let found = false;
  for (let z = 0; z < pwidth; z += 50) {
    if (x + pwidth / 2 >= z && x + pwidth / 2 <= z + 50) {
      found = true;
      break;
    }
  }
  if (found) {
    if (x < 0) {
      x = 0;
    } else {
      x = z - pwidth / 2;
    }
  }
  return x;
}

createPlayer() {
  const startX = (Math.ceil(this.gamewidth / this.GRID_SIZE) / 2) * this.GRID_SIZE;
  const startY = (this.ROWS - 1) * this.GRID_SIZE; // bottom row

  this.player = this.add.circle(startX + this.GRID_SIZE / 2, startY + this.GRID_SIZE / 2,
    this.GRID_SIZE / 2 - 4, 0x556b2f);
  this.player.setDepth(1000); // olive should always be seen
  this.gameContainer.add(this.player);

  this.playerGridX = Math.floor(startX / this.GRID_SIZE);
  this.playerGridY = this.ROWS - 1;
  this.playerOnLog = null; // track which log olive is on
}
```

The log-player mechanic was also inspired by the GitHub repository (left), where the player moves with the log after collision and dies if carried off-screen. In my version (right), this is adapted in my movePlayerWithLog function.

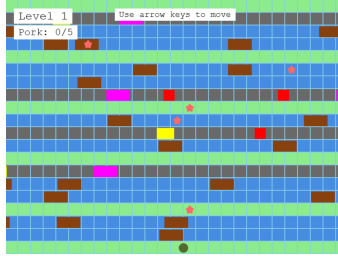
```
} else if (terrain[i] == "W") {
  if (i == 1 & 3) {
    if (cur.checkCollision(px, pwidth)) {
      let found = false;
      cur.move(dir, true);
      for (let z = cur.x; z < cur.x + cur.w; z += 50) {
        if (px + pwidth / 2 >= z && px + pwidth / 2 <= z + 50) {
          px = z;
          found = true;
          break;
        }
      }
      if (px + pwidth < 0 || px > sWidth) {
        gameState = "end" + prevGameState[0];
        deathMessage = "Rode Log too Long";
        break;
      }
    } else {
      } else if (terrain[i] == "W") {
        if (i == 1 & 3) {
          if (cur.checkCollision(px, pwidth)) {
            let found = false;
            cur.move(dir, true);
            for (let z = cur.x; z < cur.x + cur.w; z += 50) {
              if (px + pwidth / 2 >= z && px + pwidth / 2 <= z + 50) {
                px = z;
                found = true;
                break;
              }
            }
            if (px + pwidth < 0 || px > sWidth) {
              gameState = "end" + prevGameState[0];
              deathMessage = "Rode Log too Long";
              break;
            }
          } else {
            }
          }
        }
      }
    }
  }
}

movePlayerWithLog(delta) {
  if (this.playerOnLog && !this.canMove) return;

  const log = this.playerOnLog;
  const cols = Math.ceil(this.gamewidth / this.GRID_SIZE);
  this.playerGridX = log.direction * log.speed * delta / 1000; // move olive based on log position
  this.player.x = this.playerGridX * this.GRID_SIZE + this.GRID_SIZE / 2;
  // check if olive fell off the log (moved outside log bounds)
  const logLeft = log.gridX * log.width / 2;
  const logRight = log.gridX * log.width / 2;

  // death menu
  if (this.playerGridX < logLeft || this.playerGridX > logRight) {
    this.die("Fell off the log!");
    return;
  }
  if (this.playerGridX < 0.5 || this.playerGridX > cols - 0.5) {
    this.die("Pushed off screen!");
  }
}
```

Below is a basic prototype of my game, showing early versions of things such as cars, logs, and the surrounding landscape. Features like car or log movement, win/lose scenarios, and item collection have also been added. I'm still working on weird quirks with cars and logs where they will overlap visually and kill the player randomly.



Sabrina's Accomplishments

I have begun learning the basics of game development using Phaser on CodeCademy. I am about 18% done with the course which will also teach me how to do physics, animations, and visual effects. For the Mexico minigame, I found code that I can base most of my game off of ([ToastGame](#)). This code was originally written with HTML, JavaScript, and CSS which had to be translated into Phaser.

In the original code, the hitboxes would often lag behind the targets making them harder to click on. By tweaking some of the code, I was able to adjust the hitboxes so that it lined up with the targets on the screen.

```
const pinata = this.add.rectangle(startX, startY, 40, 40, 0xDEB887);
pinata.setInteractive();
pinata.on('pointerdown', () => this.onPinataClicked(pinata));
```

The original code ended the game once the player runs out of lives, but for our purposes, I changed the code to create a timed game with a duration of two minutes.

```
class MexicoScene extends Phaser.Scene {
  constructor() {
    super('MexicoScene');
    this.score = 0;
    this.timeLeft = 120;
    this.levelDuration = 120;
    this.gameOver = false;
    this.pinatas = [];
  }
}
```

Additionally, I added code to create the chiles to make the game more difficult. If the chiles were clicked on, the player's score decreases by one (the amount the score decreases may be changed in the future for added difficulty).

```
onPinataClicked(pinataEl) {
  if (this.gameOver) return;

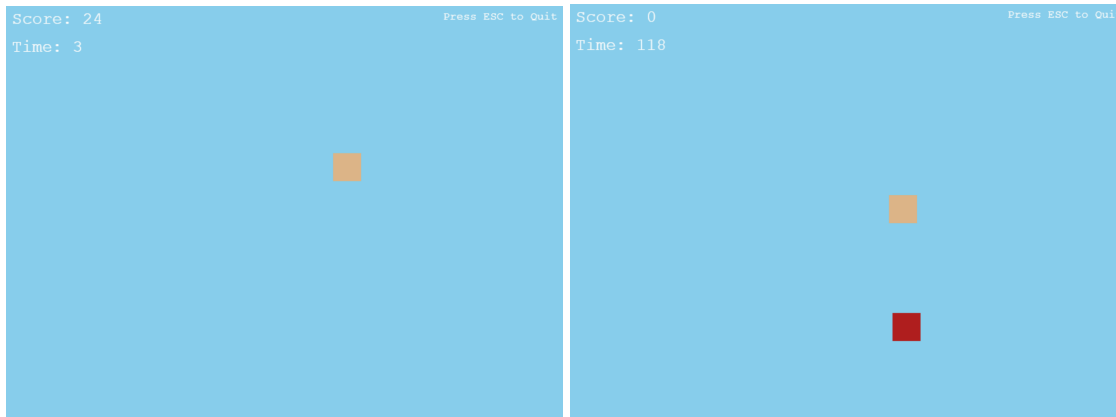
  const index = this.pinatas.findIndex(p => p.el === pinataEl);
  if (index !== -1 && !this.pinatas[index].clicked) {
    this.pinatas[index].clicked = true;
    this.score += 1;
    this.scoreText.setText('Score: ' + this.score);

    this.tweens.add({
      targets: pinataEl,
      y: -50,
      alpha: 0,
      duration: 500,
      onComplete: () => {
        pinataEl.destroy();
        const removeIndex = this.pinatas.findIndex(p => p.el === pinataEl);
        if (removeIndex !== -1) {
          this.pinatas.splice(removeIndex, 1);
        }
      }
    });
  }
}
```

By the end of the two minutes, the player will be given a score depending on how many targets they successfully hit.



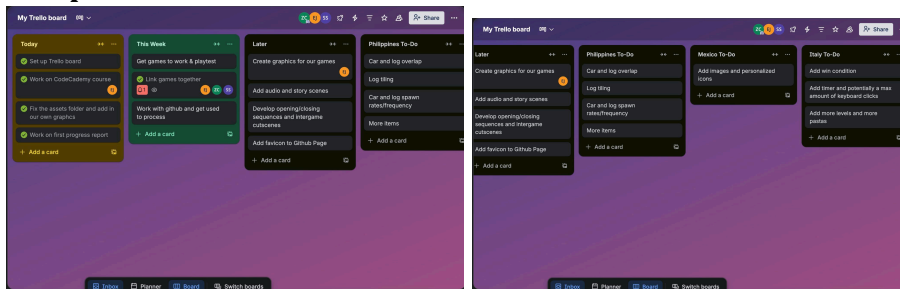
So far, the game consists of basic visuals that will be changed later on to fit the theme.



4. Difficulties Encountered this Progress Period

One of the most difficult things we faced during this progress period was using GitHub. It definitely had a learning curve. Pulling things from the right GitHub branch and learning how commit changes was difficult, but after looking into tutorials, we were able to understand it and we all work with VS Code and GitHub efficiently. Another difficulty that we encountered was time management. Sometimes we got too caught up on a minute detail that could be tweaked later when more pressing matters needed to be dealt with. We got around this by keeping each other accountable and making sure we were always working on the topic. Finally, getting acquainted with Phaser, especially when we were unfamiliar with normal JavaScript syntax was a very big hurdle, but after making more progress with the CodeCademy courses, it has become more intuitive for us.

5. Updated Trello Board and Discussion



The board was recently created, and we are continuing to work on our tasks. We have general tasks and tasks specific to each of our minigames.

6. Tasks to Be Worked on in Next Progress Period

The next progress period will be very productive. Here is a breakdown of who will do what:

Zac:

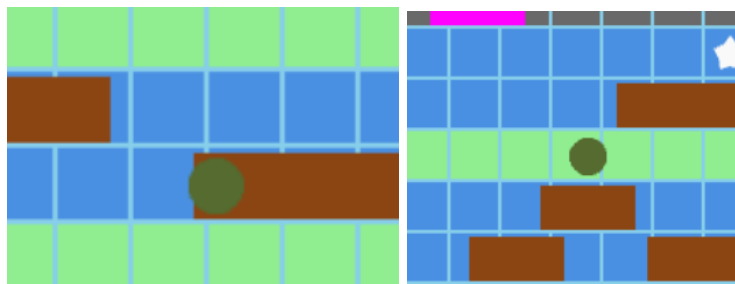
During this next project period, I will be working on expanding my game to become playable. Because my minigame is a puzzle game, the content will be very different from that of Ella or Sabrina's games. The next thing I plan to do is create a function to check if the play has won. I will do this by having the game return the rotation index of each pasta shape, which will be checked against a correct rotation index. This will result in the game returning a win. I also plan to add a limit of moves you can make on the board before you lose. This will encourage smart moves instead of moves done just to play around. Finally, I also hope to be at least 60% complete with the CodeCademy course by the end of this progress period. A lot of the later content in the course is more applicable to physics based games, but I still would like to learn more about Phaser.

Ella:

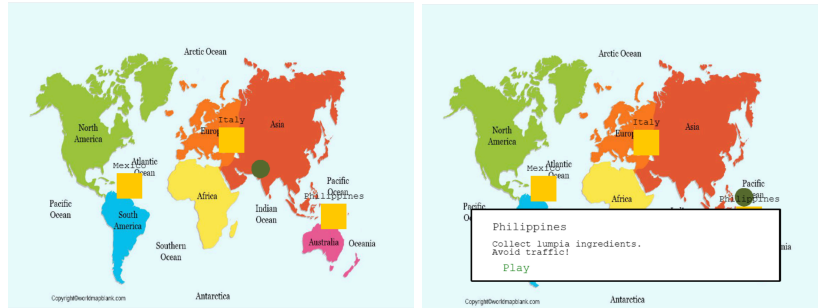
In the next project period, I want to fix the overlap of cars in my game, because when I replace the placeholders, it will create a weird visual effect.



I also want to fix the logs and tiling. When a player goes on a log, they don't snap automatically to the tiles on the log and end up in off-center places which can make the player off-center when on land as well.



Additionally, I would also like to make more progress with the Phaser CodeCademy course because the later modules on visual effects might be useful if I get around to making a 2nd game. I also want to make better graphics for my game and update the graphics for the map game selection screen. Finally, if time permits, I want to look into adding a universal currency into all games and adding a shop feature.



Sabrina:

For the next update, I plan to add in the images for the pinatas and chiles and improve the games aesthetics overall. I also want to start increasing the game's difficulty as the time goes on by spawning multiple pinatas or chiles at a time. Lastly, although the game keeps track of the score, I need to implement a scoring system that will determine whether the player has passed the game. While working on the game, I will continue working on completing the CodeCademy course. If time permits, I also want to add in animations for when the pinata and chiles are clicked on.

7. Additional Information

So far, we are very satisfied with how far we have gotten. If we keep at this pace, we should be able to add in the three extra games we hoped to create. AI coding assistance has been a great help and blended seamlessly with our work.
